Licenciatura en Sistemas de Información

Programación Avanzada - 2026

TRABAJO PRÁCTICO N.º 4

Sesiones de pago y webhook Stripe

Estudiante: Molinari Eugenia

Profesor: Mg. Lic. Ernesto Ledesma

Cátedra: Programación avanzada

Año: 2026

1. Introducción

En este trabajo práctico se desarrolló un microservicio HTTP utilizando **NestJS**, integrado con **Stripe en modo de prueba**.

El microservicio permite crear sesiones de pago mediante una API REST y recibir notificaciones de Stripe a través de un webhook cuando un pago se completa.

La implementación incluye la validación de los datos recibidos, la creación de una Checkout Session, las rutas de éxito y cancelación, y la verificación de la firma de los eventos enviados por Stripe.

2. Creación de una sesión de pago

Se implementó el siguiente endpoint:

POST /payments/create-payment-session

Este endpoint recibe el identificador de la orden (`orderId`), la moneda (`currency`) y los productos incluidos en la operación (`items`).

Los precios son convertidos a centavos antes de ser enviados a Stripe mediante `Math.round(price * 100)`. Además, el `orderId` se almacena en los metadatos del `PaymentIntent`.

Como respuesta, Stripe genera una sesión de Checkout que incluye su identificador y la URL necesaria para acceder al proceso de pago.

Evidencia – Creación de sesión de pago

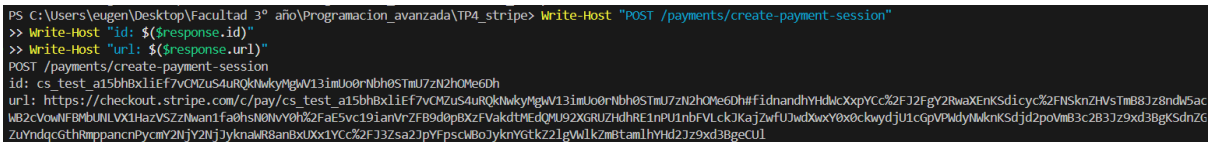


Figura 1. Creación de una sesión de pago y URL de Stripe Checkout.

3. Validación de datos y rutas de redirección

Para validar los datos recibidos se utilizaron DTOs junto con **class-validator** y **class-transformer**. También se configuró un `ValidationPipe` global con las opciones `whitelist` y `forbidNonWhitelisted`.

Se verificó que `orderId` y `currency` sean obligatorios, que exista al menos un elemento en `items` y que los precios y cantidades sean positivos. Durante las pruebas, las solicitudes con datos incorrectos fueron rechazadas con código **HTTP 400 Bad Request**.

También se implementaron las rutas:

GET /payments/success
GET /payments/cancel

Las URLs correspondientes se configuran mediante variables de entorno y no se reciben desde el cuerpo de la solicitud.

4. Implementación y prueba del webhook

Para recibir las notificaciones de Stripe se implementó:

POST /payments/webhook

El webhook utiliza el cuerpo crudo de la petición (`rawBody`) y el encabezado `stripe-signature`. La firma se verifica utilizando `stripe.webhooks.constructEvent()` junto con `STRIPE_ENDPOINT_SECRET`.

También se comprobó que una solicitud con una firma inválida sea rechazada con código **HTTP 400 Bad Request**, evitando que un evento no autorizado sea interpretado como un pago confirmado.

Para las pruebas locales se utilizó **Stripe CLI**, reenviando los eventos de Stripe al webhook del microservicio.

Luego de completar un pago de prueba, se recibió correctamente el evento `charge.succeeded`. A partir de este evento se recuperó el `orderId` asociado al pago y se mostró en los logs de la aplicación.

Evidencia – Recepción del webhook

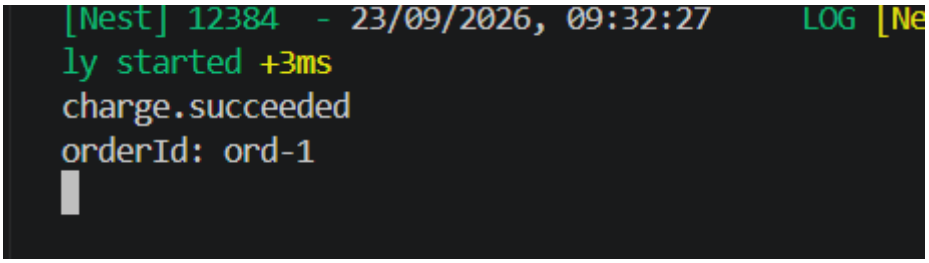


Figura 2. Recepción del evento `charge.succeeded` y recuperación del `orderId`.

5. Variables de entorno y seguridad

La configuración de Stripe se realizó mediante variables de entorno:

`PORT`
`STRIPE_SECRET`
`STRIPE_SUCCESS_URL`
`STRIPE_CANCEL_UR`
`STRIPE_ENDPOINT_SECRET`

El archivo `.env` contiene las credenciales utilizadas localmente y se encuentra excluido del repositorio mediante `.gitignore`.

En el repositorio se incluye `.env.template` sin claves privadas, permitiendo conocer las variables necesarias para ejecutar el proyecto sin exponer información sensible.

6. Repositorio

El código fuente del trabajo se encuentra dentro de la carpeta **TP4_stripe** del repositorio de Programación Avanzada:

https://github.com/EugeniaMolinari1/Programacion_avanzada/tree/main/TP4_stripe